
Universidad de las Fuerzas Armadas ESPE

Departamento: Ciencias de la Computación

Carrera: Ingeniería de Software

Tema: Principios SOLID

Prueba N°: 1-P2

Información General

- **Asignatura:** Análisis y Diseño de Software
 - **Apellidos y nombres de los estudiantes:**
Panchez Jacome Darwin Javier
Robalino Curay Cristian Isaak
Proaño Vásquez José Ignacio
 - **NRC:** 23305
 - **Fecha de realización:** 07/07/2025
-

Objetivos

- Analizar cómo SRP establece que una clase debe tener una única razón para cambiar, enfocándose en una sola responsabilidad.
- Refactorizar una clase monolítica (como Estudiante) en componentes separados (Modelo, DAO, Controlador, Vista) para cumplir con SRP y mejorar la mantenibilidad del código.

Ejemplo de SRP (Single Responsibility Principle) en Java PATRÓN SOLID

1. ¿Qué es SRP?

SRP significa Single Responsibility Principle (Principio de Responsabilidad Única), el cual establece que una clase o módulo debe tener una única razón para cambiar, enfocándose en una sola responsabilidad, y que todas sus funcionalidades deben estar alineadas exclusivamente con dicha responsabilidad.

2. Ejemplo que NO cumple SRP

La clase Estudiante presentada no cumple con el Principio de Responsabilidad Única (SRP) porque concentra múltiples responsabilidades en una sola clase. No solo modela los datos del estudiante

(atributos como id, nombre, apellido, edad), sino que también maneja la persistencia de datos simulando operaciones en una base de datos, gestiona la presentación de la información en consola mediante impresiones detalladas y controla la lógica de la interfaz de usuario a través de menús e interacciones por consola. Esta combinación de responsabilidades hace que la clase sea difícil de mantener, escalar o reutilizar de forma modular.

Código ejemplo

```
public class Estudiante {
    // ===== RESPONSABILIDAD 1: MODELAR DATOS =====
    private int id;
    private String nombre;
    private String apellido;
    private int edad;

    // Simulación de almacenamiento en memoria
    private static List<Estudiante> estudiantes = new
    ArrayList<>();

    public Estudiante(int id, String nombre, String apellido,
    int edad) {
        this.id = id;
        this.nombre = nombre;
        this.apellido = apellido;
        this.edad = edad;
    }

    // Getters y Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre =
    nombre; }
    public String getApellido() { return apellido; }
    public void setApellido(String apellido) { this.apellido =
    apellido; }
    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }

    // ===== RESPONSABILIDAD 2: PERSISTENCIA EN
    BD =====
    public void guardarEnBD() {
        estudiantes.add(this);
        System.out.println("Conectando a la base de datos...");
        System.out.println("INSERT INTO estudiantes VALUES (" +
    id + ", '" + nombre + "', '" + apellido + "', " + edad + ")");
        System.out.println("Estudiante guardado en la BD
    exitosamente");
        System.out.println("Cerrando conexión a BD");
    }

    public void actualizarEnBD() {
        System.out.println("Conectando a la base de datos...");
```

```

        System.out.println("UPDATE estudiantes SET nombre='" +
nombre + "', apellido='" + apellido + "', edad=" + edad + "
WHERE id=" + id);
        System.out.println("Estudiante actualizado
exitosamente");
        System.out.println("Cerrando conexión a BD");
    }

    public void eliminarDeBD() {
        estudiantes.removeIf(e -> e.getId() == this.id);
        System.out.println("Conectando a la base de datos...");
        System.out.println("DELETE FROM estudiantes WHERE id=" +
id);
        System.out.println("Estudiante eliminado exitosamente");
        System.out.println("Cerrando conexión a BD");
    }

    // ===== RESPONSABILIDAD 3: PRESENTACIÓN DE
DATOS =====
    public void imprimir() {
        System.out.println("=====");
        System.out.println("INFORMACIÓN DEL ESTUDIANTE");

        System.out.println("=====");
        System.out.println("ID: " + id);
        System.out.println("Nombre: " + nombre);
        System.out.println("Apellido: " + apellido);
        System.out.println("Edad: " + edad);

        System.out.println("=====");
    }

    public void imprimirResumen() {
        System.out.println("ID: " + id + " - " + nombre + " " +
apellido + " - " + edad + " años");
    }

    // ===== RESPONSABILIDAD 4: LÓGICA DE
NEGOCIO =====
    public boolean validarDatos() {
        if (id <= 0) {
            System.out.println("Error: El ID debe ser mayor a
0");
            return false;
        }
        if (nombre == null || nombre.trim().isEmpty()) {
            System.out.println("Error: El nombre no puede estar
vacío");
            return false;
        }
        if (apellido == null || apellido.trim().isEmpty()) {
            System.out.println("Error: El apellido no puede
estar vacío");
            return false;
        }
    }

```

```

        if (edad < 15 || edad > 100) {
            System.out.println("Error: La edad debe estar entre
15 y 100 años");
            return false;
        }
        return true;
    }

    public void crearYGuardarEstudiante() {
        if (validarDatos()) {
            guardarEnBD();
            imprimir();
            System.out.println("Operación completada
exitosamente");
        }
    }

    // ===== RESPONSABILIDAD 5: INTERFAZ DE
USUARIO =====
    public static void mostrarMenu() {
        Scanner scanner = new Scanner(System.in);
        int opcion;
        do {
            System.out.println("\n" + "=".repeat(50));
            System.out.println("    SISTEMA DE ESTUDIANTES (SRP
MALO) ");
            System.out.println("=".repeat(50));
            System.out.println("1. Crear estudiante");
            System.out.println("2. Mostrar todos los
estudiantes");
            System.out.println("3. Mostrar estudiante por ID");
            System.out.println("4. Actualizar estudiante");
            System.out.println("5. Eliminar estudiante");
            System.out.println("0. Salir");
            System.out.println("=".repeat(50));
            System.out.print("Seleccione una opción: ");

            opcion = scanner.nextInt();
            scanner.nextLine(); // Limpiar buffer

            switch (opcion) {
                case 1:
                    crearEstudianteDesdeMenu(scanner);
                    break;
                case 2:
                    mostrarTodosEstudiantes();
                    break;
                case 3:
                    mostrarEstudiantePorId(scanner);
                    break;
                case 4:
                    actualizarEstudianteDesdeMenu(scanner);
                    break;
                case 5:
                    eliminarEstudianteDesdeMenu(scanner);
                    break;
            }
        } while (opcion != 0);
    }

```

```

        case 0:
            System.out.println("\nHasta luego!");
            break;
        default:
            System.out.println("Opción inválida");
    }
} while (opcion != 0);

scanner.close();
}

private static void crearEstudianteDesdeMenu(Scanner scanner) {
    System.out.println("\n--- CREAR NUEVO ESTUDIANTE ---");
    System.out.print("ID: ");
    int id = scanner.nextInt();
    scanner.nextLine();
    System.out.print("Nombre: ");
    String nombre = scanner.nextLine();
    System.out.print("Apellido: ");
    String apellido = scanner.nextLine();
    System.out.print("Edad: ");
    int edad = scanner.nextInt();

    Estudiante estudiante = new Estudiante(id, nombre,
apellido, edad);
    estudiante.crearYGuardarEstudiante();

    System.out.println("\nPresione Enter para
continuar...");
    scanner.nextLine();
}

private static void mostrarTodosEstudiantes() {
    System.out.println("\n--- LISTA DE ESTUDIANTES ---");
    if (estudiantes.isEmpty()) {
        System.out.println("No hay estudiantes
registrados.");
    } else {
        for (Estudiante e : estudiantes) {
            e.imprimirResumen();
        }
    }
    System.out.println("\nPresione Enter para
continuar...");
    try { System.in.read(); } catch (Exception ex) {}
}

private static void mostrarEstudiantePorId(Scanner scanner)
{
    System.out.println("\n--- MOSTRAR ESTUDIANTE POR ID
---");
    System.out.print("ID del estudiante: ");
    int id = scanner.nextInt();
    scanner.nextLine();
}

```

```

        Estudiante estudiante = buscarPorId(id);
        if (estudiante == null) {
            System.out.println("Estudiante no encontrado.");
        } else {
            estudiante.imprimir();
        }
        System.out.println("\nPresione Enter para
continuar...");
        scanner.nextLine();
    }

    private static void actualizarEstudianteDesdeMenu(Scanner
scanner) {
        System.out.println("\n--- ACTUALIZAR ESTUDIANTE ---");
        System.out.print("ID del estudiante a actualizar: ");
        int id = scanner.nextInt();
        scanner.nextLine();

        Estudiante estudiante = buscarPorId(id);
        if (estudiante == null) {
            System.out.println("Estudiante no encontrado.");
            System.out.println("\nPresione Enter para
continuar...");
            scanner.nextLine();
            return;
        }

        System.out.print("Nuevo nombre: ");
        String nombre = scanner.nextLine();
        System.out.print("Nuevo apellido: ");
        String apellido = scanner.nextLine();
        System.out.print("Nueva edad: ");
        int edad = scanner.nextInt();

        estudiante.setNombre(nombre);
        estudiante.setApellido(apellido);
        estudiante.setEdad(edad);

        if (estudiante.validarDatos()) {
            estudiante.actualizarEnBD();
            estudiante.imprimir();
        }

        System.out.println("\nPresione Enter para
continuar...");
        scanner.nextLine();
    }

    private static void eliminarEstudianteDesdeMenu(Scanner
scanner) {
        System.out.println("\n--- ELIMINAR ESTUDIANTE ---");
        System.out.print("ID del estudiante a eliminar: ");
        int id = scanner.nextInt();
        scanner.nextLine();

        Estudiante estudiante = buscarPorId(id);

```

```

        if (estudiante == null) {
            System.out.println("Estudiante no encontrado.");
            System.out.println("\nPresione Enter para
continuar...");
            scanner.nextLine();
            return;
        }

        System.out.println("\nEstudiante a eliminar:");
        estudiante.imprimirResumen();
        estudiante.eliminarDeBD();

        System.out.println("\nPresione Enter para
continuar...");
        scanner.nextLine();
    }

    private static Estudiante buscarPorId(int id) {
        for (Estudiante e : estudiantes) {
            if (e.getId() == id) return e;
        }
        return null;
    }
}

```

3. Análisis del Problema

Esta clase tiene 4 responsabilidades distintas

1. Modelar la entidad Estudiante (atributos, constructor, getters y setters).
2. Persistencia de datos (guardar, actualizar y eliminar en base de datos).
3. Presentación de la información (métodos para imprimir y mostrar datos en consola).
4. Interfaz de usuario (menú de opciones e interacción con el usuario).

4. Ejemplo que Sí cumple SRP

Ahora se separan las responsabilidades en clases distintas:

1. Clase Modelo (solo datos):

```

public class Estudiante {
    // Atributos del modelo
    private int id;
    private String nombre;
    private String apellido;
    private int edad;

    public Estudiante(int id, String nombre, String apellido, int edad) {
        this.id = id;
    }
}

```

```

        this.nombre = nombre;
        this.apellido = apellido;
        this.edad = edad;
    }

    // Getters y setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getApellido() { return apellido; }
    public void setApellido(String apellido) { this.apellido = apellido; }
    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }
}

```

2. Clase DAO (persistencia):

```

package dao;

import model.Estudiante;
import java.util.ArrayList;
import java.util.List;

/**
 * Clase que SOLO maneja la persistencia de estudiantes (en memoria).
 * No contiene lógica de negocio, presentación ni modelo.
 */
public class EstudianteDAO {
    private List<Estudiante> estudiantes = new ArrayList<>();

    public void agregar(Estudiante estudiante) {
        estudiantes.add(estudiante);
    }

    public List<Estudiante> obtenerTodos() {
        return estudiantes;
    }

    public Estudiante buscarPorId(int id) {
        for (Estudiante e : estudiantes) {
            if (e.getId() == id) return e;
        }
        return null;
    }

    public boolean actualizar(Estudiante estudiante) {

```



```

        Estudiante e = buscarPorId(estudiante.getId());
        if (e != null) {
            e.setNombre(estudiante.getNombre());
            e.setApellido(estudiante.getApellido());
            e.setEdad(estudiante.getEdad());
            return true;
        }
        return false;
    }

    public boolean eliminar(int id) {
        Estudiante e = buscarPorId(id);
        if (e != null) {
            estudiantes.remove(e);
            return true;
        }
        return false;
    }
}

```

3. Clase Controller

```

package controller;

import dao.EstudianteDAO;
import model.Estudiante;
import java.util.List;

/**
 * Clase que SOLO maneja la lógica de negocio y coordinación.
 * Valida datos y coordina entre DAO y otras capas.
 */
public class EstudianteController {
    private EstudianteDAO dao;

    public EstudianteController(EstudianteDAO dao) {
        this.dao = dao;
    }

    // Lógica de negocio: crear estudiante solo si los datos son válidos
    public boolean crearEstudiante(int id, String nombre, String apellido, int edad) {
        if (!validarDatos(id, nombre, apellido, edad)) return false;
        dao.agregar(new Estudiante(id, nombre, apellido, edad));
        return true;
    }
}

```

```

public List<Estudiante> obtenerTodos() {
    return dao.obtenerTodos();
}

public Estudiante buscarPorId(int id) {
    return dao.buscarPorId(id);
}

public boolean actualizarEstudiante(int id, String nombre, String apellido,
int edad) {
    if (!validarDatos(id, nombre, apellido, edad)) return false;
    return dao.actualizar(new Estudiante(id, nombre, apellido, edad));
}

public boolean eliminarEstudiante(int id) {
    return dao.eliminar(id);
}

// Validación de datos (lógica de negocio)
private boolean validarDatos(int id, String nombre, String apellido, int
edad) {
    if (id <= 0) {
        System.out.println("Error: El ID debe ser mayor a 0");
        return false;
    }
    if (nombre == null || nombre.trim().isEmpty()) {
        System.out.println("Error: El nombre no puede estar vacío");
        return false;
    }
    if (apellido == null || apellido.trim().isEmpty()) {
        System.out.println("Error: El apellido no puede estar vacío");
        return false;
    }
    if (edad < 15 || edad > 100) {
        System.out.println("Error: La edad debe estar entre 15 y 100 años");
        return false;
    }
    return true;
}
}

```

4. Clase Vista (presentación):

```

package ui;

```

```
import controller.EstudianteController;

import dao.EstudianteDAO;

import model.Estudiante;

import java.util.Scanner;

/**
 * Clase que SOLO maneja la interfaz de usuario (menú y entrada de datos).
 * No contiene lógica de negocio, persistencia ni modelo.
 */

public class EstudianteUI {

    private EstudianteController controller;

    private EstudiantePrinter printer;

    public EstudianteUI() {

        this.controller = new EstudianteController(new EstudianteDAO());

        this.printer = new EstudiantePrinter();

    }

    public void mostrarMenu() {

        Scanner scanner = new Scanner(System.in);

        int opcion;

        do {

            System.out.println("\n" + "=".repeat(50));

            System.out.println("    SISTEMA DE ESTUDIANTES (SRP BUENO) ");

            System.out.println("=".repeat(50));

        } while (opcion != 0);

    }

}
```

```
System.out.println("1. Crear estudiante");

System.out.println("2. Mostrar todos los estudiantes");

System.out.println("3. Mostrar estudiante por ID");

System.out.println("4. Actualizar estudiante");

System.out.println("5. Eliminar estudiante");

System.out.println("0. Salir");

System.out.println("=".repeat(50));

System.out.print("Seleccione una opción: ");

opcion = scanner.nextInt();

scanner.nextLine();

switch (opcion) {

    case 1:

        crearEstudiante(scanner);

        break;

    case 2:

        printer.imprimirLista(controller.obtenerTodos());

        System.out.println("\nPresione Enter para continuar...");

        scanner.nextLine();

        break;

    case 3:

        mostrarEstudiantePorId(scanner);

        break;

    case 4:

        actualizarEstudiante(scanner);
```

```

        break;

        case 5:

            eliminarEstudiante(scanner);

            break;

        case 0:

            System.out.println("\nHasta luego!");

            break;

        default:

            System.out.println("Opción inválida");

    }

} while (opcion != 0);

scanner.close();

}

private void crearEstudiante(Scanner scanner) {

    System.out.println("\n--- CREAR NUEVO ESTUDIANTE ---");

    System.out.print("ID: ");

    int id = scanner.nextInt();

    scanner.nextLine();

    System.out.print("Nombre: ");

    String nombre = scanner.nextLine();

    System.out.print("Apellido: ");

    String apellido = scanner.nextLine();

    System.out.print("Edad: ");

    int edad = scanner.nextInt();

```

```
        if (controller.crearEstudiante(id, nombre, apellido, edad)) {  
            System.out.println("Estudiante creado exitosamente.");  
        }  
  
        System.out.println("\nPresione Enter para continuar...");  
  
        scanner.nextLine();  
    }  
}
```

```
private void mostrarEstudiantePorId(Scanner scanner) {  
  
    System.out.println("\n--- MOSTRAR ESTUDIANTE POR ID ---");  
  
    System.out.print("ID del estudiante: ");  
  
    int id = scanner.nextInt();  
  
    scanner.nextLine();  
  
    Estudiante estudiante = controller.buscarPorId(id);  
  
    if (estudiante == null) {  
        System.out.println("Estudiante no encontrado.");  
    } else {  
        printer.imprimir(estudiante);  
    }  
  
    System.out.println("\nPresione Enter para continuar...");  
  
    scanner.nextLine();  
}  
}
```

```
private void actualizarEstudiante(Scanner scanner) {  
  
    System.out.println("\n--- ACTUALIZAR ESTUDIANTE ---");
```

```
        System.out.print("ID del estudiante a actualizar: ");

        int id = scanner.nextInt();

        scanner.nextLine();

        Estudiante estudiante = controller.buscarPorId(id);

        if (estudiante == null) {

            System.out.println("Estudiante no encontrado.");

            System.out.println("\nPresione Enter para continuar...");

            scanner.nextLine();

            return;

        }

        System.out.print("Nuevo nombre: ");

        String nombre = scanner.nextLine();

        System.out.print("Nuevo apellido: ");

        String apellido = scanner.nextLine();

        System.out.print("Nueva edad: ");

        int edad = scanner.nextInt();

        if (controller.actualizarEstudiante(id, nombre, apellido, edad)) {

            System.out.println("Estudiante actualizado exitosamente.");

        }

        System.out.println("\nPresione Enter para continuar...");

        scanner.nextLine();

    }
}
```

```

private void eliminarEstudiante(Scanner scanner) {

    System.out.println("\n--- ELIMINAR ESTUDIANTE ---");

    System.out.print("ID del estudiante a eliminar: ");

    int id = scanner.nextInt();

    scanner.nextLine();

    Estudiante estudiante = controller.buscarPorId(id);

    if (estudiante == null) {

        System.out.println("Estudiante no encontrado.");

        System.out.println("\nPresione Enter para continuar...");

        scanner.nextLine();

        return;

    }

    printer.imprimirResumen(estudiante);

    if (controller.eliminarEstudiante(id)) {

        System.out.println("Estudiante eliminado exitosamente.");

    }

    System.out.println("\nPresione Enter para continuar...");

    scanner.nextLine();

}
}

```

5. Ventajas de aplicar SRP

- a. Si cambias cómo se imprimen los datos, no necesitas modificar la clase Estudiante ni el controlador, solo EstudiantePrinter.
- b. Si cambias el mecanismo de almacenamiento (por ejemplo, pasar de memoria a base de datos real), solo modificas o reemplazas EstudianteDAO, sin afectar ni al modelo ni a la lógica de negocio.

- c. Cada clase tiene UNA SOLA razón para cambiar, lo que mejora la mantenibilidad, la reusabilidad y el bajo acoplamiento entre componentes.
- d. Al estar separadas las responsabilidades, puedes escribir pruebas unitarias aisladas para cada parte: pruebas de validación en el controlador, pruebas de persistencia en DAO, y pruebas de presentación en el printer, sin interferencia entre ellas.
- e. El código resulta más modular y escalable: si mañana se implementa una interfaz gráfica (GUI) o API web, solo se reemplaza EstudianteUI sin modificar el modelo ni la lógica de negocio.
- f. La lectura y comprensión del sistema es inmediata: un desarrollador nuevo puede identificar fácilmente qué clase hace qué tarea (datos, almacenamiento, control o interfaz).
- g. La coordinación queda clara: el controlador se encarga exclusivamente de validar datos y delegar tareas a DAO y UI, sin romper el principio de una única responsabilidad.

Conclusiones

- El ejemplo refactorizado demuestra que cada clase tiene una tarea específica (ej. persistencia en EstudianteDAO, presentación en EstudiantePrinter), lo que facilita la reutilización y las pruebas unitarias.
- Al separar responsabilidades en clases distintas, el código se vuelve más modular, fácil de mantener y escalar, reduciendo el riesgo de errores al realizar cambios futuros.

Recomendaciones

- No concentrar múltiples responsabilidades (lógica, persistencia, UI) en una sola clase. En su lugar, dividir funcionalidades según SRP para seguir buenas prácticas de diseño.
- Combinar SRP con otros principios SOLID (como Inversión de Dependencias) o patrones como MVC (Modelo-Vista-Controlador) para estructurar proyectos complejos de manera eficiente.

Rúbrica de Evaluation

Criterio	Puntaje Máximo
Explica con claridad qué es SRP	4 puntos
Identifica correctamente las responsabilidades mezcladas	4 puntos
Muestra correctamente el ejemplo que sí cumple SRP	4 puntos
Analiza ventajas de aplicar SRP	4 puntos
Presenta el contenido de forma clara y ordenada	4 puntos
TOTAL SOBER 20 PTOS	

